Reproducibility: execution record and application integrity

Untagged PDF derivative | HTML reader

Reproducibility: execution record and application integrity

This section is a machine-verifiable reproducibility certificate. Every value below is computed by the analysis pipeline and injected at render time, establishing a chain of custody from configuration through code to publication.

Untagged PDF derivative | HTML reader

Reproducibility (part 2)

The discipline is the one that gates this project's CI: every prose number is a generated token, every token is emitted by one generator function,

Reproducibility (part 3)

and any drift between narrative and computed result fails the build before a green PDF exists (Peng 2011).

Determinism contract for seeded scientific results

Reproducing every reported number requires only two recorded inputs: the global seed pinned here and the software environment fingerprinted in the next subsection. The determinism contract fixes the first —

Determinism contract for... (part 2)

it states exactly what is held constant, what is asserted to machine tolerance, and what is deliberately not claimed byte-identical.

Seed. The global seed is 0, threaded through every `np.random.default_rng(seed)`; the global `np.random` state is never used.

Determinism contract for... (part 3)

Assertions. Recovery identities use exact or machine-tolerance assertions; seeded study reports are regression-tested for repeatability under the recorded software environment.

Determinism contract for... (part 4)

Rendered PDF/HTML/slide containers are validated as fresh publication products but are not claimed byte-identical across toolchain versions.

Untagged PDF derivative | HTML reader

Determinism contract for... (part 5)

Execution. No mocks are used: every test is a genuine computation on small categorical distributions or a seeded simulation under this repository's explicit no-mocks policy.

Application integrity and solver-health receipts

The labeled own-data path produces a different evidentiary object from a research report. It validates one-dimensional categorical inputs, hashes the canonical semantic request,

Application integrity and... (part 2)

delegates once to the shared aggregation domain, and records numerical health without adding a decision or interpretation.

Application integrity and... (part 3)

The application flow in fig. 22 reads from top to bottom. Panel A follows validation and canonical hashing; Panel B places the exact two-by-two `solver_status` key below the result classifier; Panel C separates the upper atomic-write row from the lower verification lane.

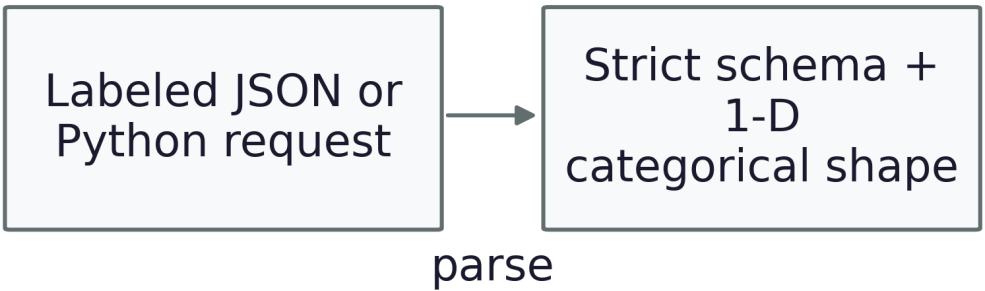
Application integrity and... (part 4)

Invalid inputs or unsafe destinations fail before directory creation. Non-nominal numerical results retain their artifacts and return a distinct status rather than being silently promoted or discarded.

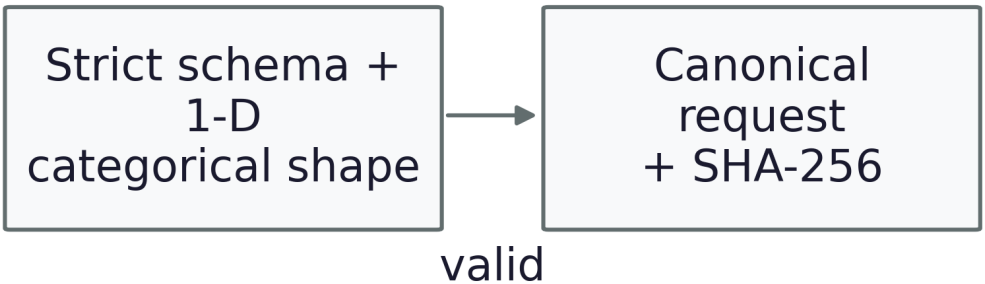
Application integrity and... (part 5)

An application receipt can verify artifact integrity and, when requested and available, source equivalence. Requiring nominal solver health adds a numerical gate. None of those levels establishes calibration, domain suitability, scientific validity, decision correctness, or acceptance.

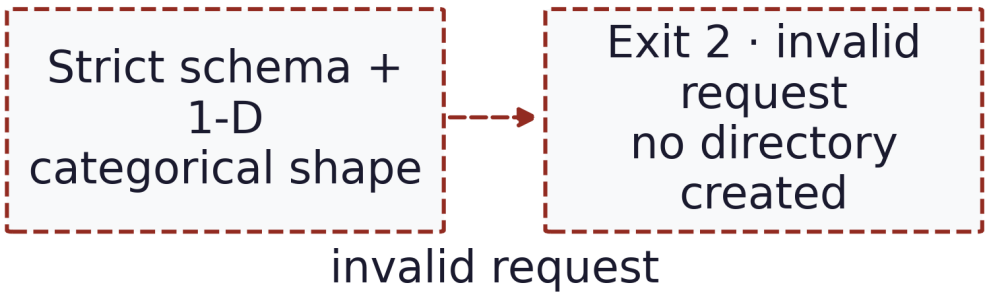
Application integrity and... (part 6)



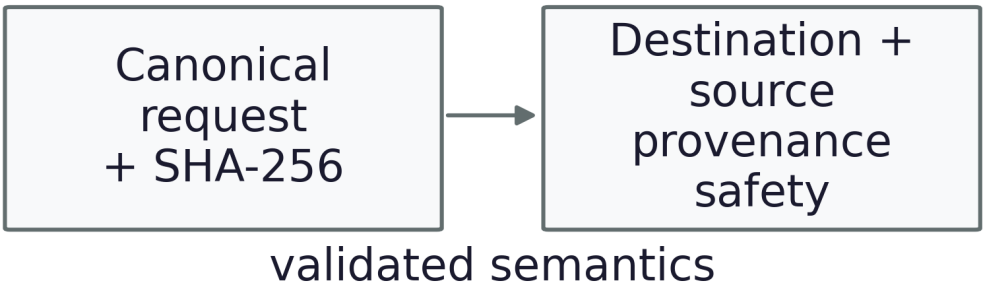
Application integrity and... (part 7)



Application integrity and… (part 8)



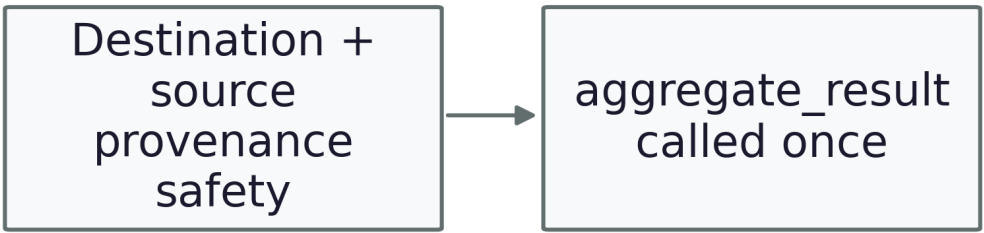
Application integrity and… (part 9)



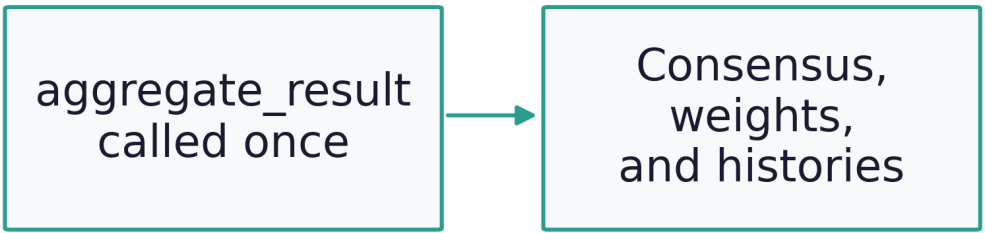
Application integrity and… (part 10)



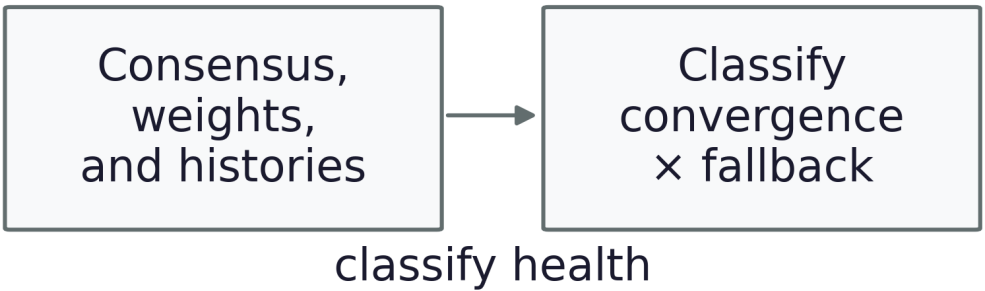
Application integrity and... (part 11)



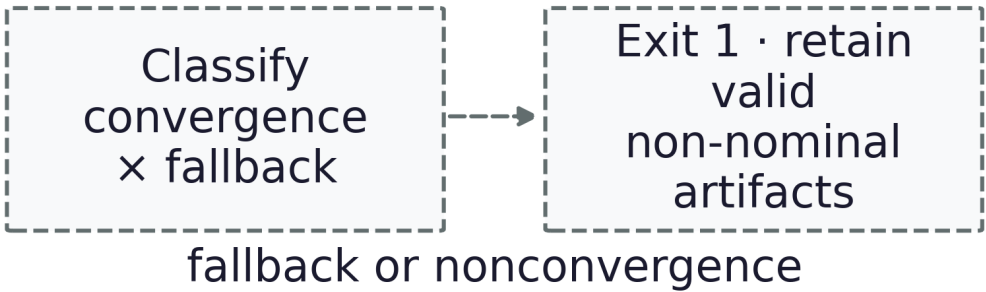
Application integrity and... (part 12)



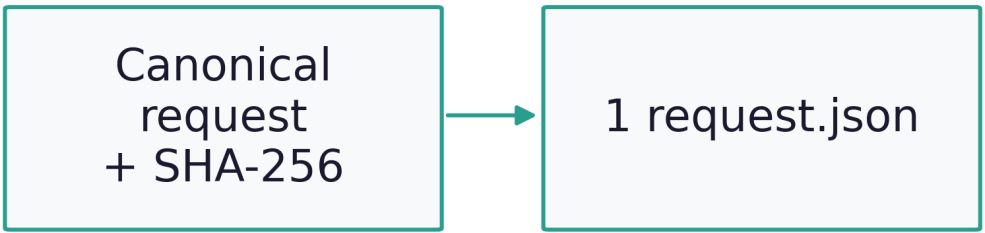
Application integrity and… (part 13)



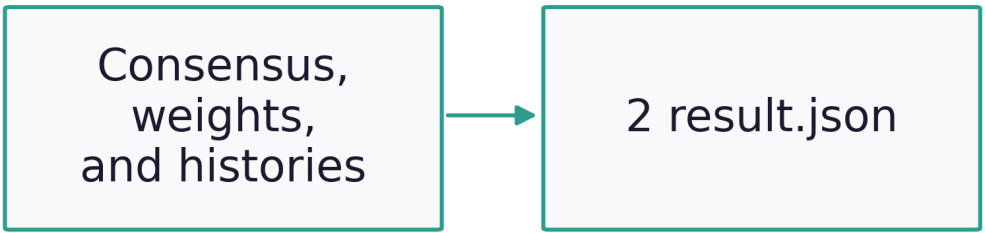
Application integrity and… (part 14)



Application integrity and... (part 15)

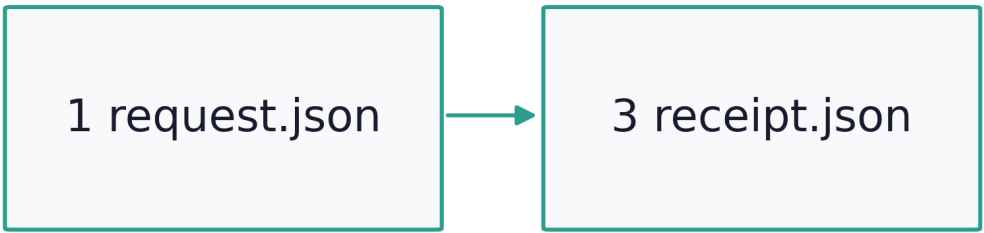


Application integrity and... (part 16)

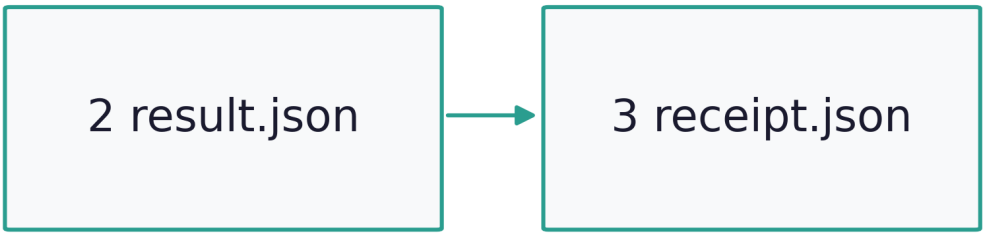


semantic output

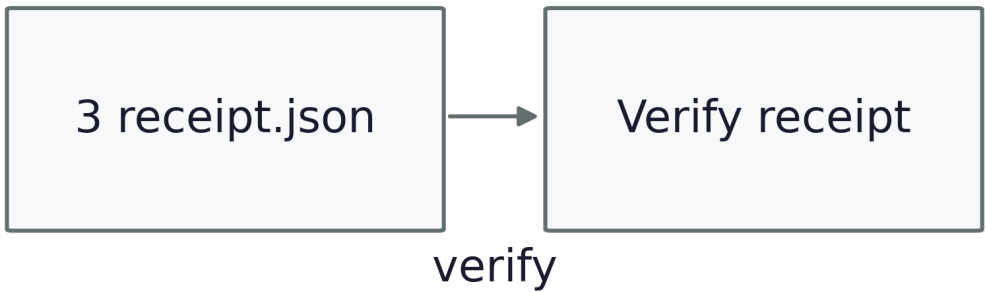
Application integrity and... (part 17)



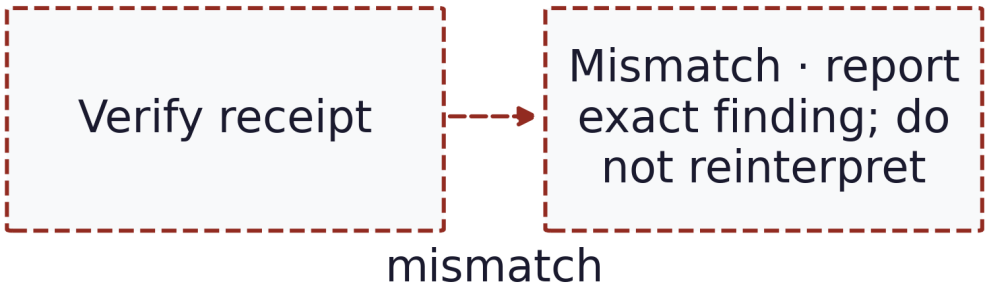
Application integrity and… (part 18)



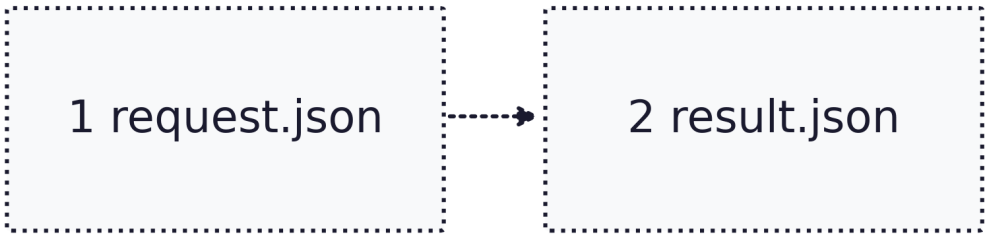
Application integrity and... (part 19)



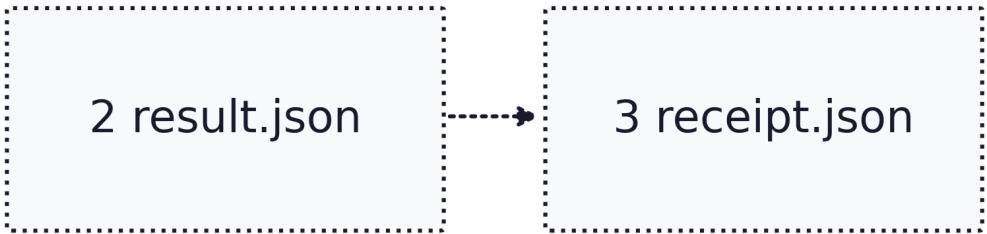
Application integrity and… (part 20)



Application integrity and... (part 21)



Application integrity and... (part 22)



Application integrity and… (part 23)

converged · no fallback
nominal

Untagged PDF derivative | HTML reader

Application integrity and... (part 24)

converged · fallback
converged_with_fallback

Application integrity and… (part 25)

not converged · no fallback
not_converged

Untagged PDF derivative | HTML reader

Application integrity and... (part 26)

not converged · fallback
not_converged_with_fallback

Application integrity and... (part 27)

Artifact integrity
Establishes: recorded request/result bytes match the receipt

Application integrity and... (part 28)

Artifact integrity
Does not establish: equivalence to another source checkout or nominal solver health

Application integrity and... (part 29)

Source equivalence
Establishes: requested source inputs match recorded source provenance

Application integrity and... (part 30)

Source equivalence
Does not establish: calibration, domain suitability, or scientific validity

Application integrity and... (part 31)

Nominal solver health
Establishes: the recorded run converged without fallback

Application integrity and... (part 32)

Nominal solver health
Does not establish: a downstream decision, acceptance, or truth of an interpretation

Application integrity and... (part 33)

Application receipt does not establish: calibration

Application integrity and... (part 34)

Application receipt does not establish: domain suitability

Untagged PDF derivative | HTML reader

Application integrity and... (part 35)

Application receipt does not establish: scientific validity

Untagged PDF derivative | HTML reader

Application integrity and... (part 36)

Application receipt does not establish:
downstream decision correctness

Untagged PDF derivative | HTML reader

Application integrity and... (part 37)

Application receipt does not establish:
acceptance

Environment fingerprint for the reported run

The second reproduction input is the exact toolchain. Every field below is captured by the successful full test-and-coverage receipt before final variable generation rather than transcribed by hand.

Environment fingerprint for... (part 2)

The receipt is bound to the source, tests, manuscript, source-owned documentation, release metadata, ISC tree, dependency lock, and fresh analysis receipt.

Environment fingerprint for... (part 3)

It rejects any pre/post-suite drift in that boundary, so a reader matching this environment and the seed above can reproduce the seeded results; the config hash lets them confirm they are running the configuration from which this manuscript was rendered.

Environment fingerprint for... (part 4)

Field	Value
Python	3.14.9
NumPy	2.4.2
SciPy	1.18.0
PyTorch (MLP complement)	2.12.1

Environment fingerprint for... (part 5)

The exact environment used for the reported run is recorded in tbl. 11.

Reader-surface accessibility boundary

The validated HTML manuscript is the canonical accessibility-enhanced reading surface. Its source gate requires a page language and title, a skip link and main landmark, non-empty image alternatives, figure captions, labelled full-size links, unique identifiers, resolved references,

Reader-surface... (part 2)

and present local assets on every generated page.

Reader-surface... (part 3)

These deterministic checks are not a claim of WCAG conformance: alternative-text quality, contrast, keyboard behavior, reading order, reflow, mathematics, and assistive-technology behavior still require manual review.

Reader-surface... (part 4)

The combined manuscript PDF is generated through the source-controlled LuaLaTeX/tagpdf path and is released only when `pdfinfo` reports `Tagged: yes`, qpdf exposes a non-empty `/Lang` and `StructTreeRoot`, and the source-bound language check passes.

Untagged PDF derivative | HTML reader

Reader-surface... (part 5)

Some Poppler builds omit the language line from `pdfinfo` even when `/Lang` is present. The separate slide PDFs are checked structurally, textually, through renderer logs during the producing run, and by raster inspection, but do not inherit the manuscript tagging status.

Reader-surface... (part 6)

Renderer logs are retained with the external verification evidence rather than the public source tree because they contain environment-local timestamps and paths.

Reader-surface... (part 7)

Tagged structure is not PDF/UA conformance: a PDF/UA claim requires a dedicated conformance report plus screen-reader and reading-order review; `qpdf` structure checks and successful text extraction alone are insufficient.

Untagged PDF derivative | HTML reader

Test and coverage evidence for the claim surface

Acceptance criteria. 287 total, 283 passing.

Test and coverage evidence... (part 2)

Project test suite. 3992 collected cases; the bound successful receipt records zero failed cases. The project no-mocks policy remains a separately executable source contract.

Test and coverage evidence... (part 3)

Combined line and branch coverage on `src/`. 92.47%, achieved by the bound full gate with branch measurement enabled. The shared coverage threshold is $\geq 90\%$ on this combined measure.

Test and coverage evidence... (part 4)

To regenerate this evidence from a clean checkout, run the project suite under the pinned development environment. Local execution and hosted CI are separate results; the hosted checks must pass for the exact reviewed commit before public integration. Both use the following coverage gate:

Test and coverage evidence... (part 5)

```
uv run --locked --extra dev pytest tests/
\
--cov=src --cov-fail-under=90
```

Test and coverage evidence... (part 6)

For a release-facing hydration, use the receipt-producing wrapper after any required provisional pre-test render, then rerun hydration without its provisional flag:

Test and coverage evidence... (part 7)

```
uv run --locked --extra dev python
scripts/validate_test_coverage.py
uv run --locked python scripts/\
z_generate_manuscript_variables.py
```

Artifact inventory for figures, data, and reports

Category	Count
Figures	1285
Data files	5
Reports	36

Artifact inventory for... (part 2)

The top-level artifact counts in tbl. 12 complement the recursive, checksum-bearing release manifest.